

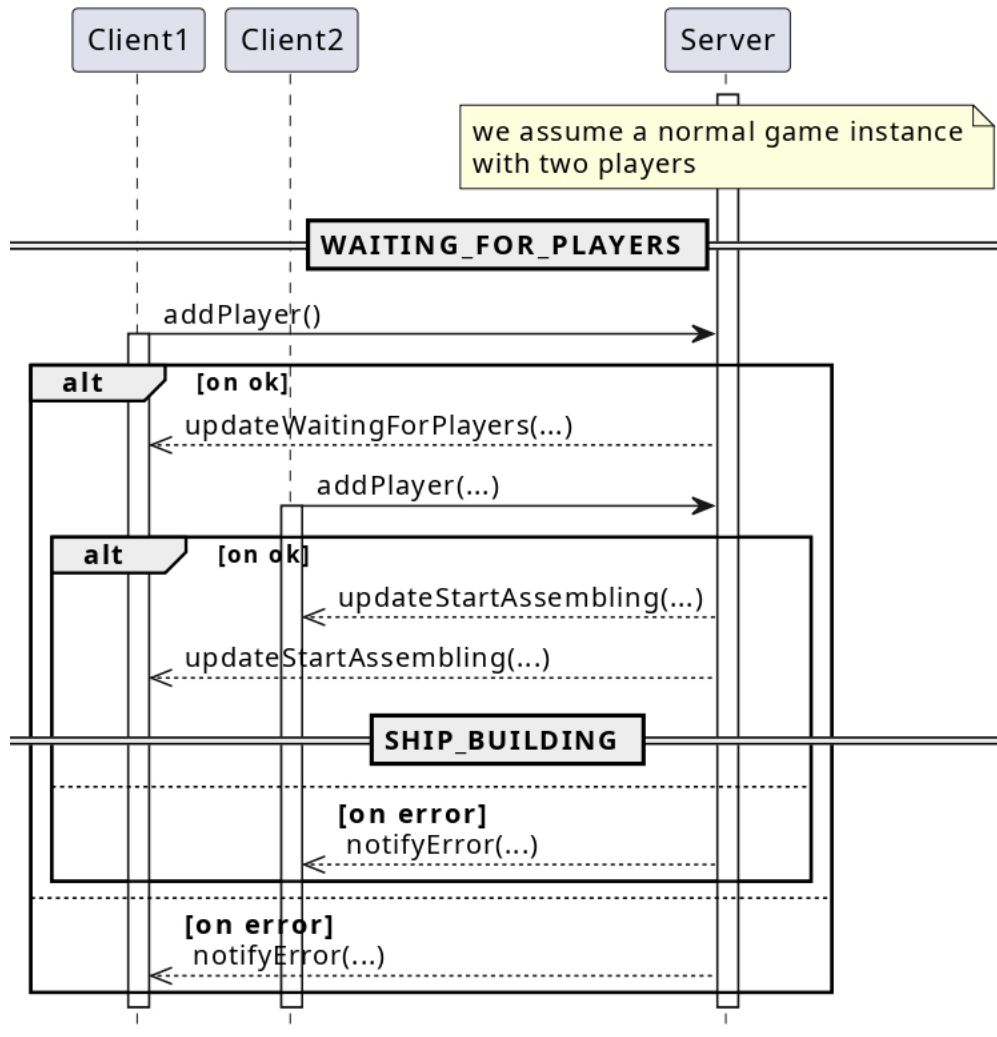
COMMUNICATION PROTOCOL

This document describes the Network Communication Protocol between clients and server designed for the *Galaxy Trucker* application. For the description of the possible interactions between the parties we have exploited UML Sequence Diagrams, to present in a more intuitive and readable manner the interfaces that clients and server must rely on. The general communication mechanism is the following:

- 1) When a player wants to perform an action, the corresponding **client** will send a request to the server. The latter, interacting with the game Controller, will modify the model accordingly. The interface used for sending requests is defined in the class *VirtualServer.java* (package *it/polimi/ingsw/galaxytrucker/network*).
- 2) Once the request has been managed by the **server**, this will send one or more updates back to the client, which will handle them by modifying the local View accordingly. The interface used for sending updates to clients is defined in the class *VirtualView.java* (package *it/polimi/ingsw/galaxytrucker/network*).

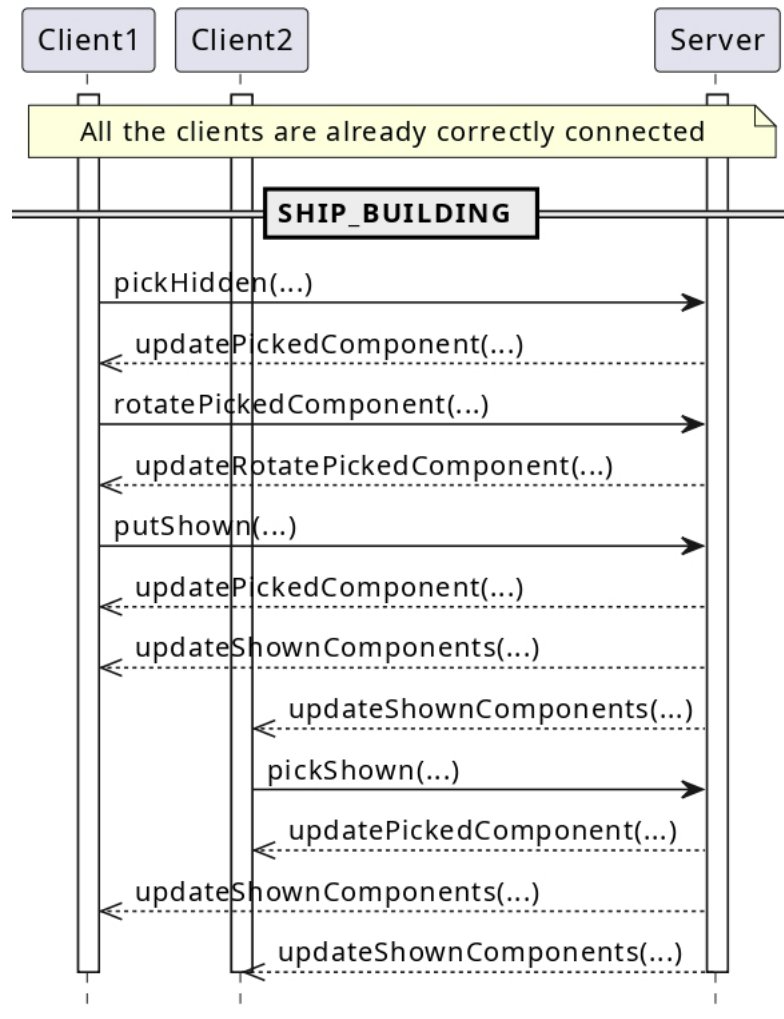
The following diagrams describe some use cases of a Galaxy Trucker game by means of Client-Server interactions.

PLAYERS ENTER THE GAME



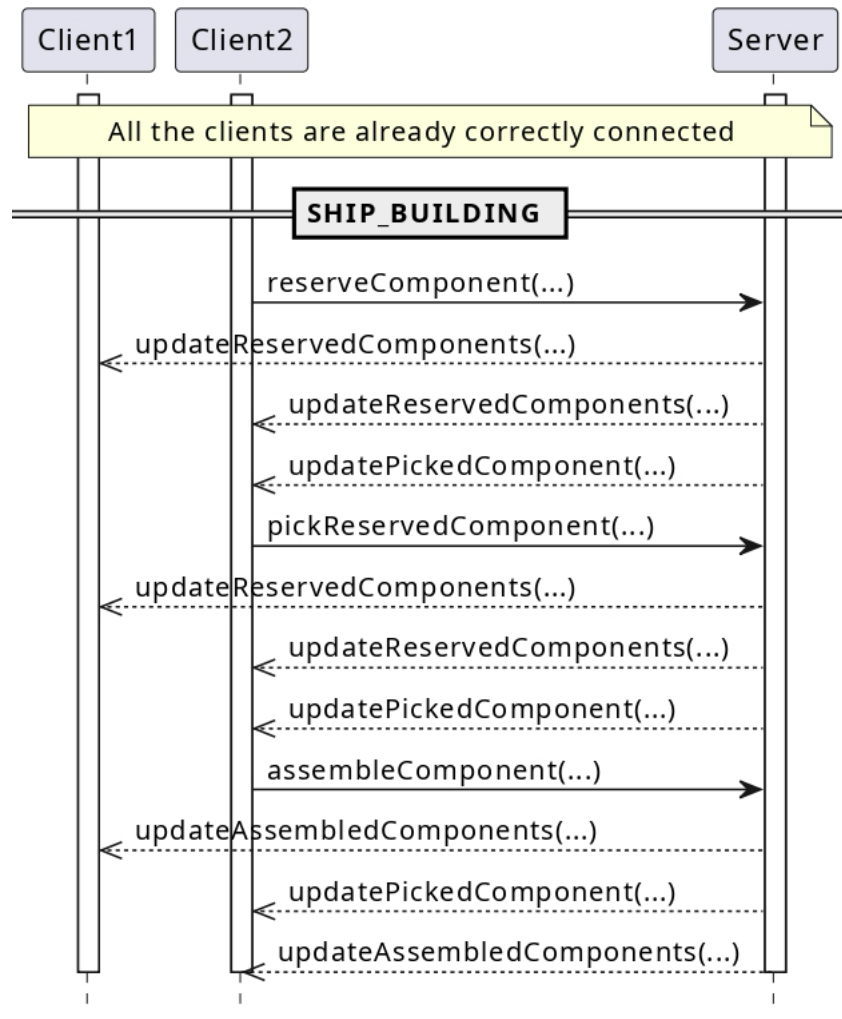
To enter the game, the clients send a request to the server specifying nickname and desired ship color. For the first player that enters, the server will send an update in such way that the client can wait for other players in a kind of lobby stage. When the last player (the number of players required is specified when a game is created) enters, the server sends an update to all clients so that the assembling phase can begin. In general, each time that a client sends a request to the server an exception can be thrown by the model (due to an illegal action/request); every time this happens, the server will send a *notifyError()* in order to provide to the players a description of the problem. **All the requests presented in the next diagrams can produce an error, but for the sake of simplicity and readability this case will not be considered.**

SHIP BUILDING (1)



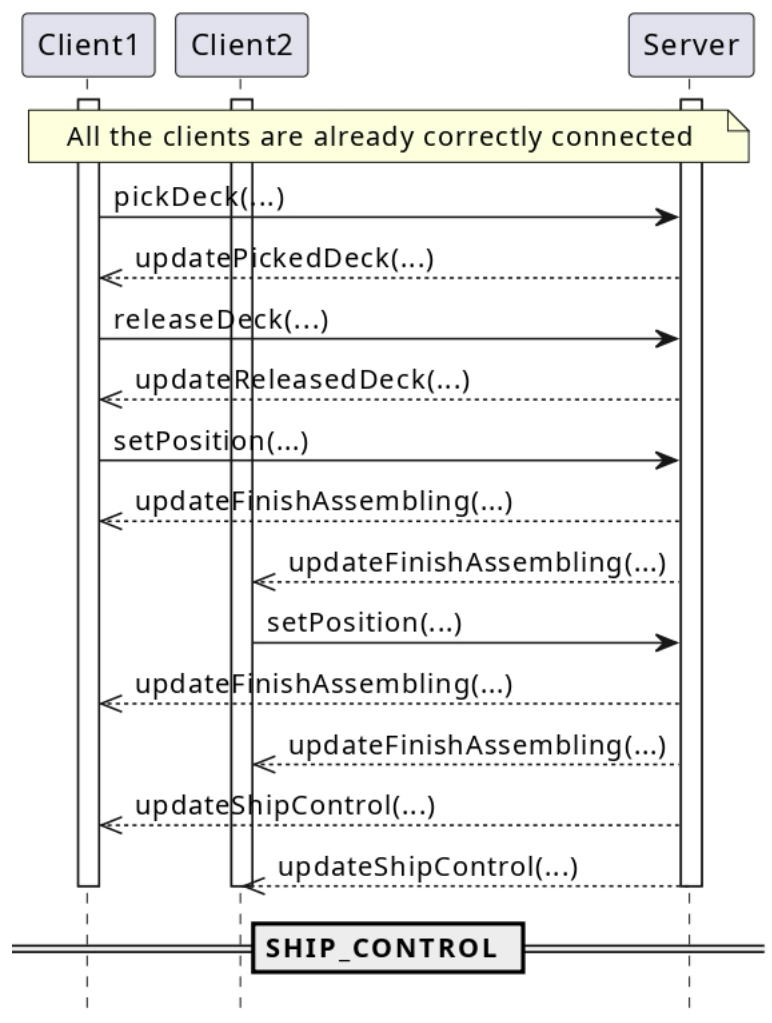
During the assembling phase, players can pick a hidden or shown component, rotate it and release it. The server will send an *updatePickedComponent()* to a client each time the component in the corresponding player's hand is released or reserved and every time the player picks a new component. Instead, when the set of shown components is changed, all the players will be notified.

SHIP BUILDING (2)



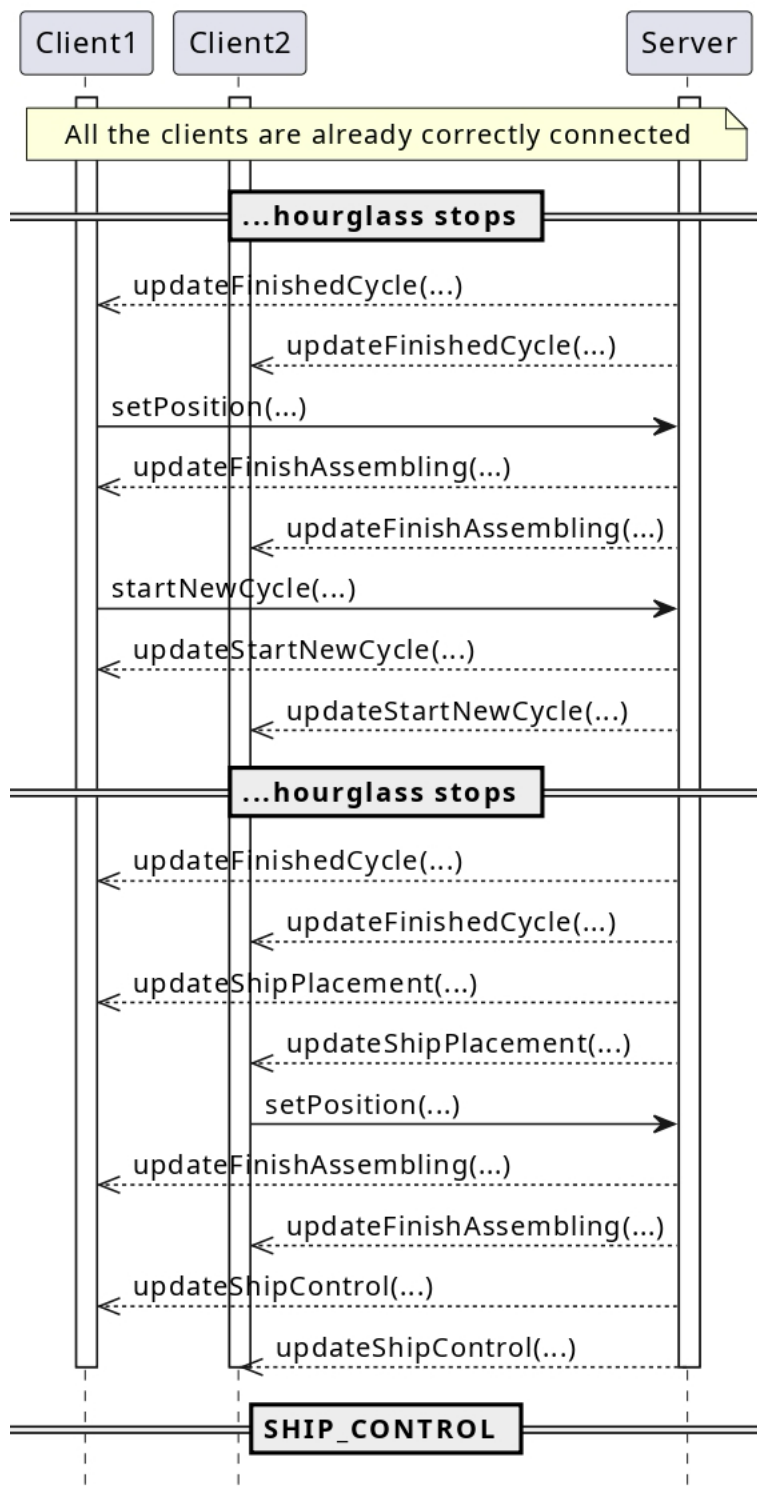
During the assembling phase, clients can also reserve the picked component or pick a reserved one. In such cases, the server will notify all clients, because **in each moment of the game each player must be able to see the state of the other players' ship boards**. The same mechanism holds also when a player assembles on its shipboard the component it has just picked.

SHIP BUILDING (3)



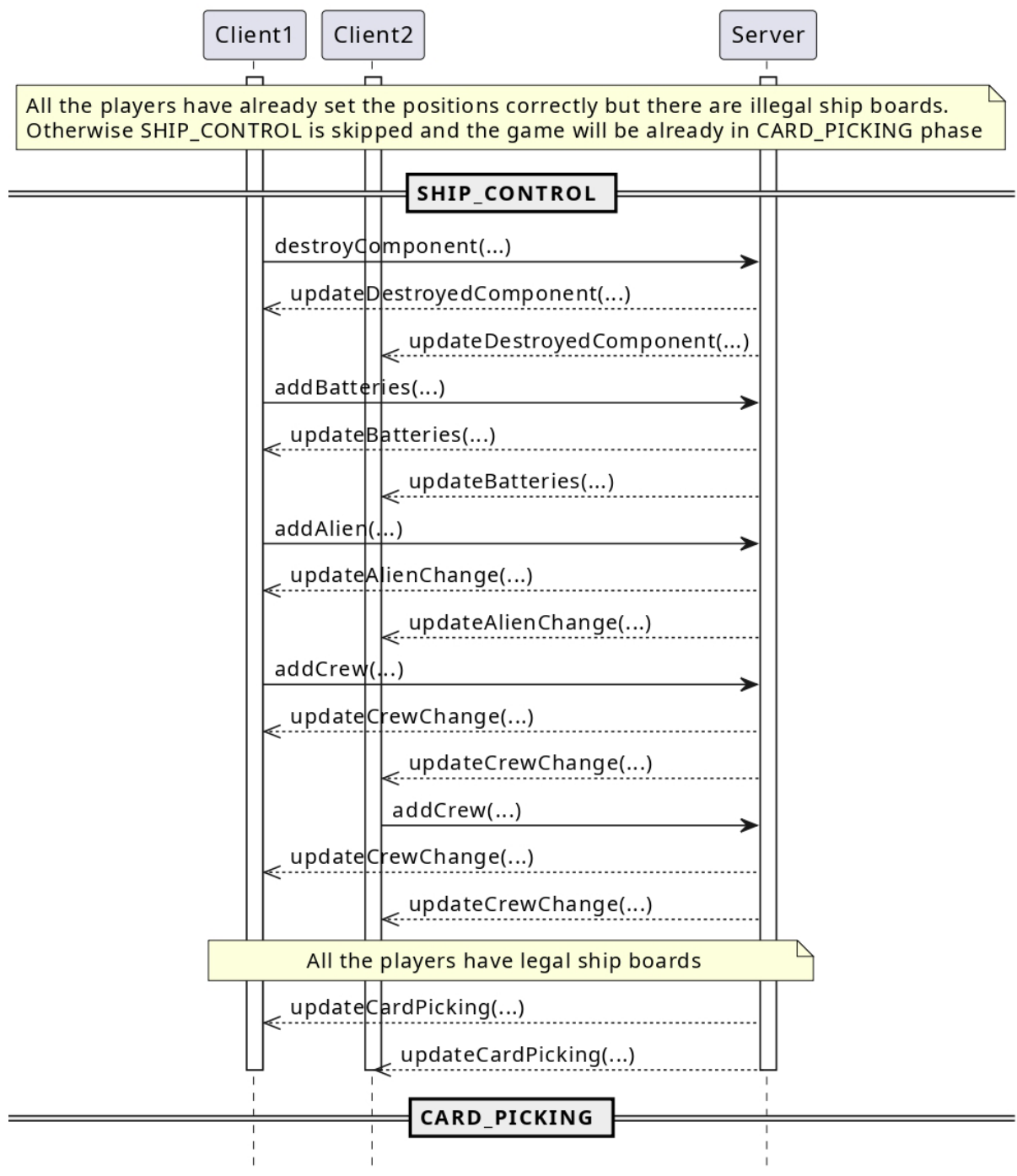
During the assembling phase, players can also pick one of the available adventure card decks and release it after viewing its content. For these actions, the server will notify only the involved client. When a player wants to stop the building phase of its shipboard, it will set its position on the flight board, action that will be notified to all clients in such way that they can always see the updated state of the flight board. When all players have placed their ship, the control phase begins and they are all notified by the server with `updateShipControl()`.

SHIP PLACEMENT



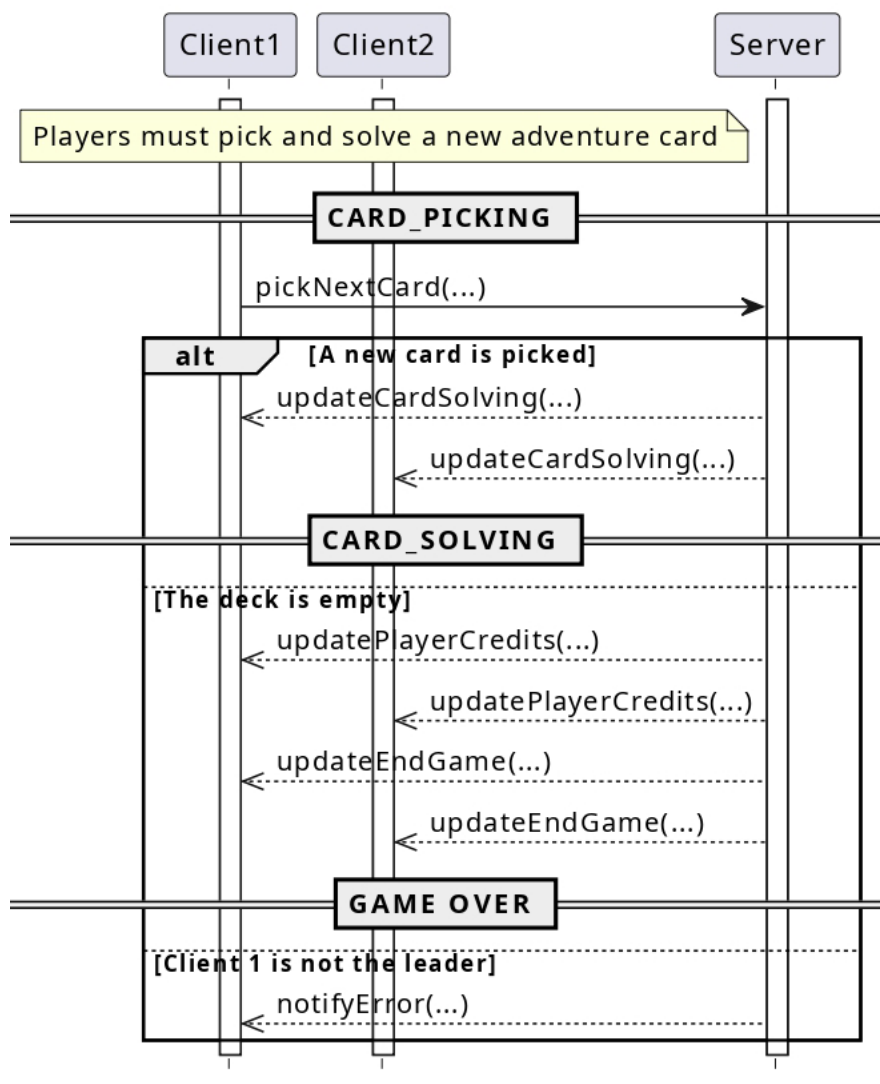
If the hourglass stops before all players have placed their ship on the flight board, the first player that finishes assembling can start the last cycle of the hourglass. All clients are notified by the server both when a cycle starts and when a cycle finishes. When the last cycle is over, the server sends an update to all players to force them to finish assembling. The placement mechanism is the same of the one in the previous diagram.

SHIP CONTROL



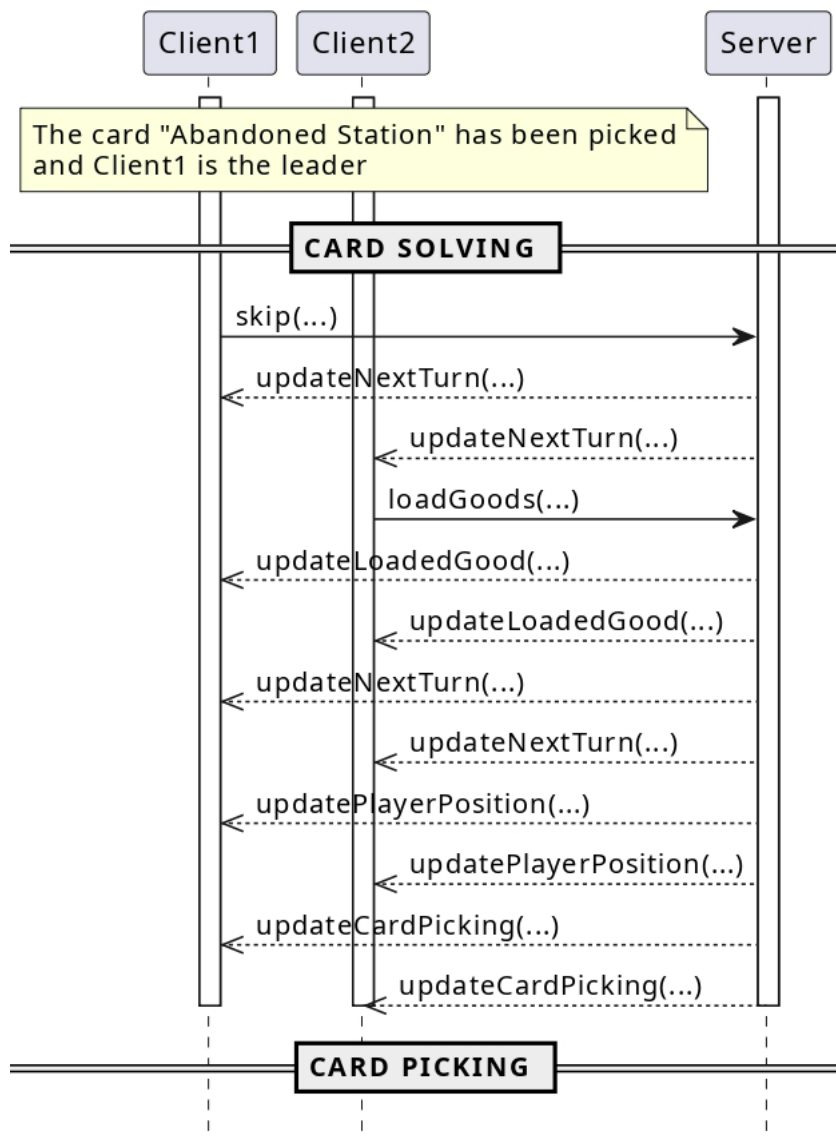
During the control phase, the players must make sure that their ships are valid, i.e. that there aren't any misplaced components, that all cabins have their crew members (or aliens eventually) and that all battery containers are full. In this phase, the server will notify all clients after every successful action performed by a player. Once all the ships are in a valid state (which is automatically verified by the server) the game enters the flight phase and all the players are notified that the first adventure card must be picked.

CARD PICKING



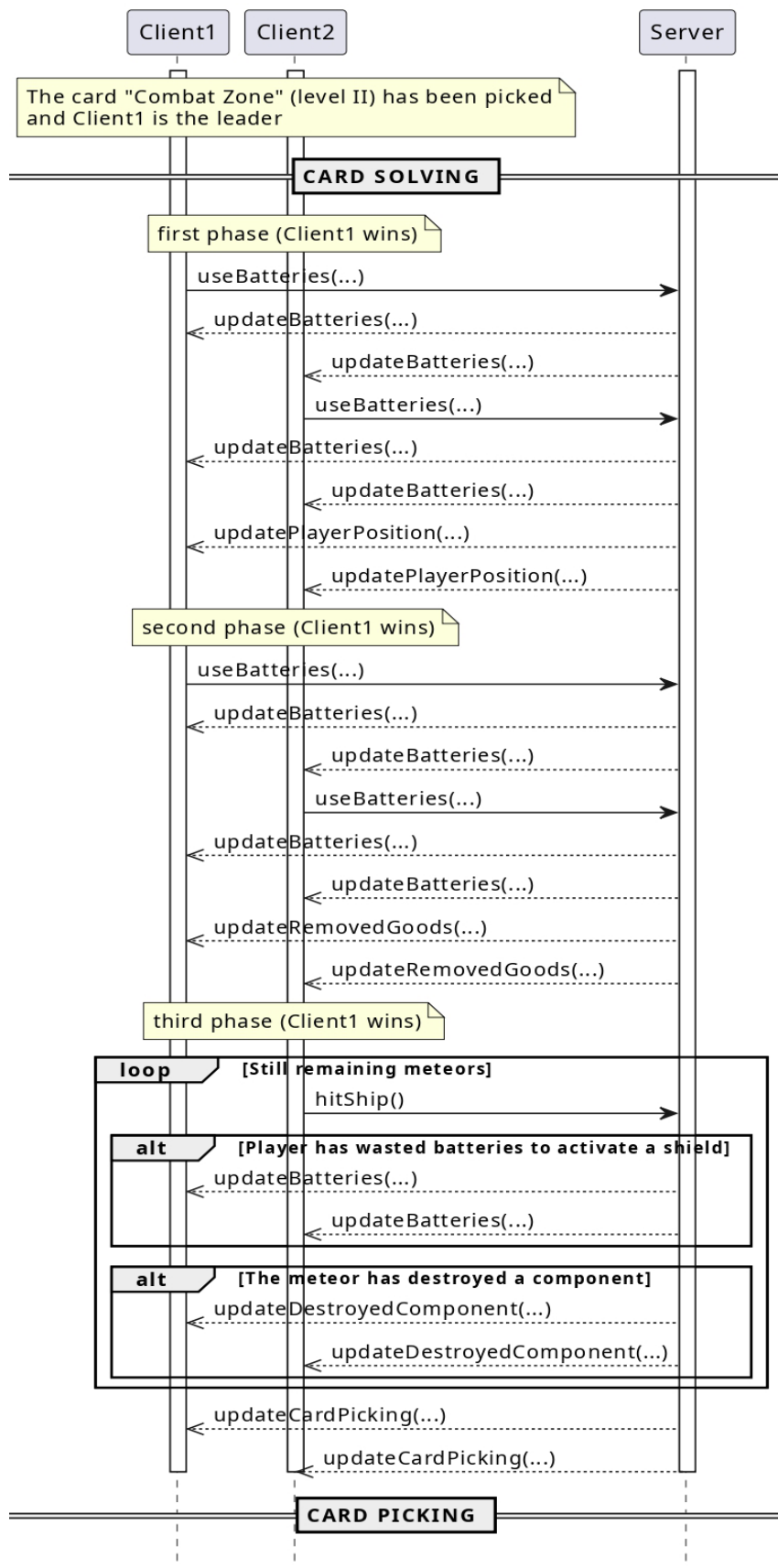
During the “card picking” phase, the leader of the game will try to pick a new card to solve (if a player that isn’t the leader tries to do so, the server will answer with an error message). If a new card is successfully picked, the game will switch to the card solving phase and all players will be notified. Instead, if the deck is empty, the game must end; the server will compute the final reward (credits) for each player and will notify them about the credits’ change and about the end of the game.

CARD SOLVING (Abandoned Station)



The diagram shows an example of interaction for the management of the “Abandoned Station” card. The first player decides not to land on the station, while instead the second player loads the station goods on its shipboard (but loses also flight days). The server will notify both players about the goods that have been charged by Client2 and about its new position on the flight board. Because the solving phase for the card is finished, the server will also send an `updateCardPicking()` to the clients. In general, **every time the game turn passes to the next player, all clients are notified so that they can always know which player must perform an action in a specific moment. For the sake of readability, this update won’t be reported in the next diagram.**

CARD SOLVING (Combat Zone Level II)

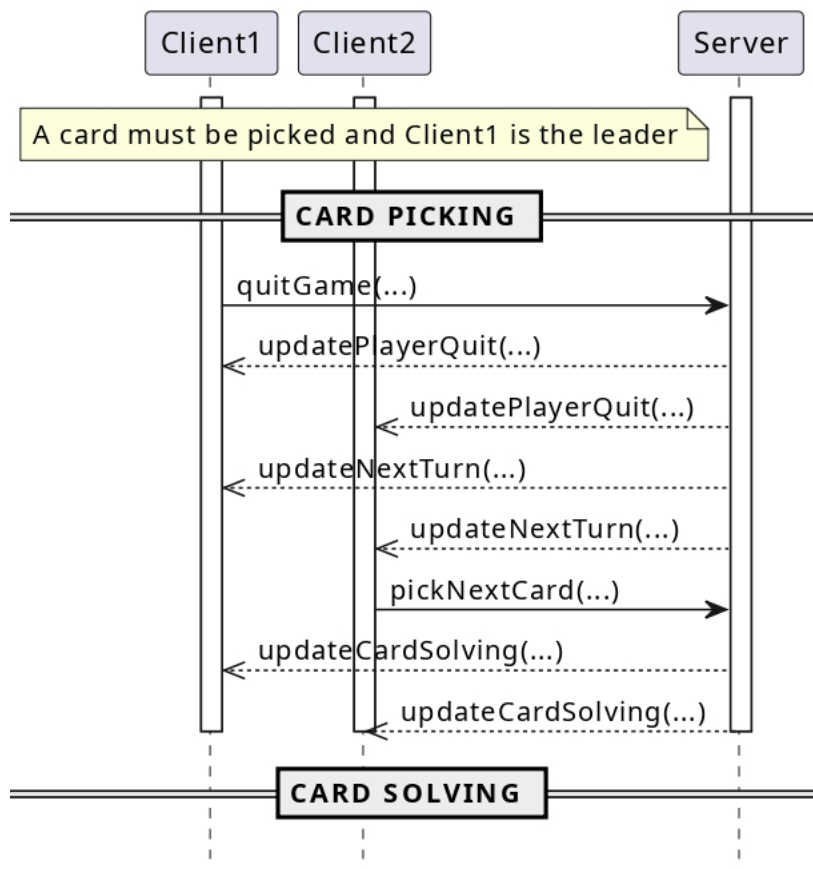


The diagram shows an example of interaction for the management of the Level II “Combat Zone” card.

- 1) **First phase:** the players declare their cannon strength by specifying the number of batteries to use to activate double cannons; the weakest player will lose a certain amount of flight days, which will be notified to all clients by the server. The server will also send specific updates regarding the usage of batteries during this phase.
- 2) **Second phase:** the players declare their engine strength by specifying the number of batteries to use to activate double engines; the weakest player will lose a certain amount of precious goods, which will be notified to all clients by the server. The server will also send specific updates regarding the usage of batteries during this phase.
- 3) **Third phase:** the player with less crew members is hit by a sequence of cannon shots. For each shot, after throwing the dice, the client will ask the server to launch the meteor by specifying the number of batteries to use to activate a shield (in general, the player must specify also the number of batteries to use to activate double cannons, because *hitShip()* can also be used for meteor shots). After each shot, the server will notify all clients about the eventual lost batteries and destroyed components for the involved player.

Once the combat zone is concluded, the game switches to the “card picking” phase and all clients are notified.

QUITTING THE GAME



Before a card is picked, players can decide whether to quit the game or not. When a player asks to quit the game, the server will notify all clients about the fact that the player has left. If the player that leaves the game is the leader, the turn will switch to the next player (new leader). In general, **during the game some circumstances (for example, no more crew, no more engine strength in open space, client disconnection) can force the server to make a player quit the game automatically; each time this happens, all clients will be notified.**

OTHER REQUESTS

- *landing()*: invoked by a client to specify the crew members that have to leave the ship; in case of success, the server will answer with a sequence of *updateCrewChange()*.
- *planetLanding()*: invoked by a client to specify the planet where to land in order to gain goods (referring to the “Planets” adventure card).
- *defeat()*: invoked when a player is fighting against an enemy (smugglers, pirates or slavers), in order to specify the number of batteries to use to activate double cannons; in case of success, the server will answer with a sequence of *updateBatteries()*.
- *fly()*: invoked by a client to specify the number of batteries to use to activate double engines (referring to the “Open Space” adventure card); in case of success, the server will answer with a sequence of *updateBatteries()*.